

CSC372 Comparative Programming Languages

ILC Rm 130, Tue / Thu 12:30pm-1:45pm

Course Description

Comparative Programming Languages (CPL) is the study of programming paradigms as embodied in high-level programming languages. The goal is to understand their principles, strengths, and limitations. The course examines how different paradigms—such as imperative-OO, scripting, functional, and logic—shape the design, understanding, and implementation of software. Through comparative analysis, CPL highlights the trade-offs between expressiveness, efficiency, and reliability, while also exploring the historical and theoretical motivations behind each paradigm, including prompting as an abstract form of PL.

Instructor and Contact Information

Roberto Giacobazzi, 713, giacobazzi@arizona.edu

Tue: 3pm-5pm and Thu: 3pm-5pm

TAs: Oghenevovwe Ikumariogbe: oaikumariogbe@arizona.edu and Ali Muaz: muaz@arizona.edu

Course Home page: <https://www2.cs.arizona.edu/people/giacobazzi/styled-9/index.html>

Instructor Home Page: <https://www2.cs.arizona.edu/people/giacobazzi/>

We will use D2L for internal communications

Course Format and Teaching Methods

The course will primarily consist of in-class lectures and practice. Small-group activities and in-class discussions will be included to help students become familiar with different programming language styles and methodologies.

Course Objectives

The course will explore the three major programming paradigms: imperative-OO-scripting, functional, and logic. Each paradigm will be introduced through a step-by-step progression, starting from simple illustrative examples and advancing toward more formal proof techniques for program correctness. The central idea is that a deep understanding of a programming paradigm requires not only learning how to write code, but also how to reason about its correctness—for instance, through type systems and assertion-based proof methods. Practicing code correctness proofs provides the theoretical foundation for designing accurate prompts for large language models (LLMs). By learning how to formally reason about program behavior and verify correctness across different paradigms, students acquire the ability to formulate precise and unambiguous specifications. This skill directly translates into the crafting of effective prompts, which must clearly encode the intended logic and constraints in order to guide LLMs toward generating correct and reliable code in a variety of programming languages, closing the gap between programming and prompting. In this respect prompting may be seen as a programming paradigm.

Expected Learning Outcomes

- You will be able to use programming languages belonging to any of the three main programming paradigms (imperative, functional and logic)
- You will be able to prove correctness for the code written in these
- You will practice in the pragmatics of programming with different languages and styles
- You will be able to engineer prompts that interact productively with AI systems for code generation.

Transferable Career Skills

National Association of Colleges and Employers (NACE) Career Readiness:

Career readiness is a foundation from which to demonstrate requisite core competencies that broadly prepare the college-educated for success in the workplace and lifelong career management. For new college graduates, career readiness is key to ensuring successful entrance into the workforce.

There are eight career readiness competencies, each of which can be demonstrated in a variety of ways." (NACE, 2025)

- Career & Self Development
- Communication
- Critical Thinking
- Equity & Inclusion
- Leadership
- Professionalism
- Teamwork
- Technology

In this course, we will focus on the following competencies:

- **Teamwork:** Students will include lectures, in-class discussions, and activities, with a strong emphasis on teamwork through group work that requires collaboration with other students both during and outside of class.
- **Technology:** Students will use different technologies, including LLM for producing reliable code.
- **Critical Thinking:** Students will be able to navigate through different programming styles acquiring an agnostic attitude towards the use of programming languages.
- **Communication:** Students are encouraged to emphasize communication by interacting with teaching assistants and using various channels, such as D2L, email, class discussions, and Piazza, to stay updated on course materials.

Makeup Policy for Students Who Register Late

Students who add or register for the course after the official start date are responsible for all material covered prior to their enrollment.

- **Missed Classes & Assignments:** Late registrants are expected to review missed lectures, readings, and assignments independently.
- **Makeup Work:** Makeup opportunities for missed quizzes, homework, or other graded activities are **not automatically granted**. The instructor may, at their discretion, provide alternative arrangements when feasible.
- **Exams:** If an exam was missed before registration was completed, no retroactive makeup will be given. Students will be evaluated on assessments administered after their enrollment date.
- **Responsibility:** It is the student's responsibility to communicate promptly with the instructor and to make every effort to catch up on course requirements.

Course Communications

The course will use official UA e-mail address, a dedicated web site and D2L for communications.

Required Texts or Readings

The course will be covered with copy of the slides. Recommended texts are:

- Maurizio Gabbrielli and Simone Martini. Programming Languages: Principles and Paradigms, Second edition. In Undergraduate Topics in Computer Science (UTiCS) Springer, 2023.

- Adam Books Webber. Modern Programming Languages: A Practical Introduction 2nd edition. Franklin, Beedle & Associates Inc. 2011.
- Leon Sterling and Udi Shapiro. The art of Prolog. MIT Press. 1986.
- OCaml Programming. <https://cs3110.github.io/textbook/cover.html>
- OCaml from the very beginning: <https://cs3110.github.io/textbook/cover.html>
- Glynn Winskel. Formal semantics of Programming Languages. MIT Press. 1993.
- Jim Blandy and Jason Orendorff. Programming Rust. O'Reilly. 2018.

Availability: purchased, library reserve, or class handouts and D2L

Assignments and Examinations: Schedule/Due Dates

There will be a mid-term exam scheduled on Oct. Thu, 16th, 2025, and a final exam as planned by the academic calendar on Wed. Dec 17th. Homework will consist of small programming projects and challenges, scheduled for the third, sixth, ninth, and twelfth weeks of the course. Students will have a maximum of two weeks to complete and submit their work.

Final Examination

Final Exam Wed. December 17th, 1pm-3pm. Here it is the link to the Final Exam Regulations and Final Exam Schedule: <https://registrar.arizona.edu/faculty-staff-resources/room-class-scheduling/schedule-classes/final-exams>

Grading Scale and Policies

The grading is based on Homeworks and Exams (mid-term and final) according to this mapping:

- Midterm exam (15%): in person exam with multiple-choice and open-response questions
- Final exam (45%) composed in the following way:
 - 15% In-person exam with multiple-choice and open-response questions on 12/17/2025
 - 30% Evaluation of a final-year project completed in groups of three to five students
- Homeworks assignments (40%). There will be 4 homework assignments, assigned typically on Tuesdays and due in no more than 2 weeks.

All roundings will be made upward.

The final project will be assigned immediately after the midterm exam. Students will have six weeks to complete it, and all projects must be submitted before the end of the course.

The final grade is obtained according to the following mapping A: 100-91, B: 90-71, C: 70-51, D: 50-41, E: 40-31, F: 30-0. Without prior arrangement, missed exams and late homework assignments are not graded for credit. In exceptional and documented circumstances extra time can be requested.

Incomplete (I) or Withdrawal (W):

Requests for incomplete (I) or withdrawal (W) must be made in accordance with University policies, which are available at <https://catalog.arizona.edu/policy/courses-credit/grading/grading-system>.

Dispute of Grade Policy:

Up to a week is the maximum time period for disputing a grade on any graded homework or exam.

Honors Credit

Students wishing to contract this course for Honors Credit should e-mail me to set up an appointment to discuss the terms of the contract and to sign the Honors Course Contract Request Form. The form is available at <http://www.honors.arizona.edu/honors-contracts>

Scheduled Topic and Activities

Week 1

- Introduction
- Abstract Machines

Week 2

- Syntax and Semantics of PL
- Syntax and Semantics of PL

Week 3

- Verification & understanding as a comparative principle
- Proof principles: Theory
- Homework 1

Week 4:

- Imperative PL: Syntax and Semantics (min-C a la Rust)
- Imperative PL: Verification and Hoare Logic (min-C a la Rust)

Week 5

- Imperative PL: Practicing verification
- Imperative PL:

Week 6

- Functional languages
- Functional languages verification
- Homework 2

Week 7

- Writing an interpreter in ML
- Writing an interpreter in ML

Week 8

- Mid Term exam
- The Rust PL

Week 9

- The Rust PL
- Rust-Verification
- Homework 3

Week 10

- Logic languages
- Logic languages

Week 11

- Logic languages: control features
- Logic languages verification

Week 12

- Writing an interpreter in Prolog
- Writing an interpreter in Prolog
- Homework 4

Week 13

- Coding by Prompting: Some theory
- Context engineering: Some experiment

Week 14

- Coding in Rust by context engineering and verification
- Coding in Rust by context engineering and verification

Week 15

- Coding in Rust by context engineering: hacking generated code
- Coding in Rust by context engineering: hacking generated code

Week 16

- Final exam

Classroom Behavior Policy

To foster a positive learning environment, students and instructors have a shared responsibility. We want a safe, welcoming, and inclusive environment where all of us feel comfortable with each other and where we can challenge ourselves to succeed. To that end, our focus is on the tasks at hand and not on extraneous activities (e.g., texting, chatting, reading a newspaper, making phone calls, web surfing, etc.).

Students are asked to refrain from disruptive conversations with people sitting around them during lecture. Students observed engaging in disruptive activity will be asked to cease this behavior. Those who continue to disrupt the class will be asked to leave lecture or discussion and may be reported to the Dean of Students.

Students are encouraged to bring laptops or tablets to class, as these devices will be integrated into in-class experiments, interactive exercises, exams, and practice activities.

Safety on Campus and in the Classroom

For a list of emergency procedures for all types of incidents, please visit the website of the Critical Incident Response Team (CIRT): <https://cirt.arizona.edu/case-emergency/overview>

Also watch the video available at

https://arizona.sabacloud.com/Saba/Web_spf/NA7P1PRD161/app/me/ledetail;spf-url=common%2Flearningeventdetail%2Fcrty0000000000003841

University-wide Policies link

Links to the following UA policies are provided here: <https://catalog.arizona.edu/syllabus-policies>

Participating in the course and attending lectures and other course events are vital to the learning process. As such, attendance is required at all lectures. Absences may affect a student's final course grade. If you anticipate being absent, are unexpectedly absent, or are unable to participate in class online activities, please contact me as soon as possible. To request a disability-related accommodation to this attendance policy, please contact the Disability Resource Center at (520) 621-3268 or drc-info@email.arizona.edu. If you are experiencing unexpected barriers to your success in your courses, the Dean of Students Office is a central support resource for all students and may be helpful. The Dean of Students Office is located in the Robert L. Nugent Building, room 100, or call 520-621-7057.

If the instructor is unavailable, the respective class will be recorded and made accessible asynchronously from home, at least 48 hours prior to the scheduled class time. A synchronous session for questions and answers will be arranged within the class hours.

Department-wide Syllabus Policies and Resources link

Links to the following departmental syllabus policies and resources are provided here, <https://www.cs.arizona.edu/cs-course-syllabus-policies> :

- Department Code of Conduct
- Illnesses and Emergencies
- Obtaining Help
- Confidentiality of Student Records
- Land Acknowledgement Statement

Subject to Change Statement

Information contained in the course syllabus, other than the grade and absence policy, may be subject to change with advance notice, as deemed appropriate by the instructor.